UNIVERSITY OF TRANSPORT AND COMMUNICATIONS
INTERNATIONAL EDUCATION FACULTY

FINAL PROJECT REPORT

DEVELOPING A FACE RECOGNITION APPLICATION USING JAVA SWING AND OPENCV

Class: **Vietnam – English Information Technology 2** Cohort: **65**

Team Members:

1. **Phung Nguyen Trong An** **242630900**
2. **Nguyen Viet Khanh** **242630950**
3. **Le Duc Tai** **242630972**

Instructor: **Dao Thi Le Thuy**

Ha Noi, 2026

TABLE OF CONTENTS

TABLE OF CONTENTS 1

LIST OF FIGURES 3

LIST OF TABLE 4

INTRODUCTION 5

CHAPTER 1: OVERVIEW OF THE PROJECT 7

1.1 Introduction to Face Recognition 7

1.2 The Role of Face Recognition in Real Life 7

1.3 Research Subjects and Research Scope 8

1.3.1 Research Subjects 8

1.3.2 Research Scope 8

1.4 Technologies Used 8

1.5 System Architecture 9

CHAPTER 2: THEORETICAL BASIS 11

2.1 Java Programming Language 11

2.2 Java Swing 11

2.3 Java Database Connectivity 11

2.4 Open Source Computer Vision Library 12

2.5 Face Detection 12

2.6 Face Recognition 13

2.7 YuNet and SFace Models 13

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN 14

3.1 System Functional Analysis 14

3.2 Database Design 15

3.3 Class Design 16
3.4 User Interface Design 17
CHAPTER 4: IMPLEMENTATION 19
4.1 Application Structure 19
4.2 MainFrame and User Interface 20
4.3 Webcam Processing and Camera Display 24
4.4 Face Detection Implementation 27
4.5 Face Recognition Implementation 28
4.6 Face Registration and Gallery Management 30
4.7 Database Connection and Recognition Log 32
4.8 Experimental Results 34
CHAPTER 5: CONCLUSION AND FUTURE DEVELOPMENT 38
5.1 Conclusion 38
5.2 Limitations of the Project 38
5.3 Future Development 39
5.4 Lessons Learned 39
REFERENCES 41

LIST OF FIGURES

Figure 1.1. Face Recognition System Workflow 10

Figure 4.1. Application Project Structur 19

Figure 4.2. MainFrame Window Initialization Code 20

Figure 4.3. Functional Panel Initialization Code 21

Figure 4.4. TabbedPane Interface Organization Code 21

Figure 4.5. Camera Control When Switching Tabs 22

Figure 4.6. Default Camera Startup for the Register Tab 22

Figure 4.7. Webcam Resource Release When Closing the Application 23

Figure 4.8. Main Interface of the Face Recognition Application 24

Figure 4.9.Webcam and Frame Initialization Code 24

Figure 4.10. Webcam Opening Validation Code 25

Figure 4.11. Real-time Webcam Processing Thread 25

Figure 4.12. Webcam Image Rendering Code 26

Figure 4.13. Webcam Stop and Resource Release Code 27

Figure 4.14. User Interface During Face Registration 27

Figure 4.15. Real-time Face Recognition Interface 29

Figure 4.16. Face Registration Interface 31

Figure 4.17. Gallery Management Interface 31

Figure 4.18. Detection Log Table in phpMyAdmin 34

Figure 4.19. Successful Face Recognition Result 36

Figure 4.20. Unknown Face Recognition Result 36

LIST OF TABLE

Table 1.1. Technologies and Their Roles in the Face Recognition Application 9

Table 3.1. Main Functions of the Face Recognition System 14

Table 3.2. Structure of the detection_log Table..... 16

Table 3.3. Main Classes of the Face Recognition Application..... 16

INTRODUCTION

1. Project Background

In recent years, information technology has developed rapidly and has been widely applied in many areas of modern life. In particular, image processing and face recognition systems are increasingly used in management, security, attendance systems, surveillance, and user authentication. Face recognition technology helps automate the process of identifying individuals through facial images, reducing manual operations and improving management efficiency.

In Java technology courses, building an application with a graphical user interface, data processing, and database connectivity is an important task that helps students strengthen their programming knowledge. The project "Development of a Face Recognition Application Using Java and OpenCV" was carried out to apply knowledge of object-oriented programming, Java Swing, file processing, JDBC, and MySQL databases to a practical problem.

The application is developed as a desktop program that allows users to use a webcam for real-time face detection and recognition. Through this project, the group had the opportunity to explore the OpenCV library, face recognition models, and methods of integrating image processing technologies into Java applications.

2. Practical Significance

This project applies Java programming knowledge to the development of a practical application involving image processing and database management. Through the implementation process, the group gained a better understanding of desktop interface development, webcam data processing, and MySQL database connectivity using JDBC.

Besides its educational value, the face recognition system can also be applied to practical problems such as student attendance management, employee management, and access control in areas requiring user authentication. Furthermore, the project serves as a foundation for developing more advanced artificial intelligence and computer vision applications in the future.

3. Report Structure:

- **Chapter 1:** Presents an overview of the project, including the background, practical significance, objectives, scope of research, and program structure.
- **Chapter 2:** introduces the theoretical background related to the project, including Java programming, Java Swing, JDBC, OpenCV, YuNet, and SFace.
- **Chapter 3:** Describes the analysis and design of the system, including database design, system workflow, and overall architecture.
- **Chapter 4:** Presents the implementation process, main program modules, application interface, and experimental results of the system.
- **Chapter 5:** Provides conclusions, limitations, and future development directions of the project.

CHAPTER 1: OVERVIEW OF THE PROJECT

1.1 Introduction to Face Recognition

In this project, face recognition is not only studied as a theoretical topic but also applied in a real desktop application. The system receives input from the webcam, detects the face region, extracts facial features, and compares them with registered data. This process helps students understand how artificial intelligence models can be combined with normal Java programming to solve a practical problem [4], [5], [6].

Face recognition is a technology that uses facial images to identify or verify an individual's identity. Face recognition systems are capable of detecting and recognizing people through images, videos, or data captured from cameras in real time [4]. Nowadays, this technology is widely applied in many fields such as security systems, attendance management, employee management, and user authentication.

1.2 The Role of Face Recognition in Real Life

Compared with manual identification, face recognition can reduce processing time and improve convenience because users do not need to remember passwords or carry cards. For example, in a classroom attendance system, students can be recognized automatically through a camera. In an office system, employees can check in faster and the manager can review attendance history more easily.

However, face recognition systems also need to be designed carefully. The accuracy of the system may be affected by lighting, camera quality, face angle, and the number of training samples. Therefore, in this project, the scope is limited to a basic environment with stable lighting and frontal faces so that the main learning objectives can be achieved.

Face recognition technology plays an important role in automation and improving the security of modern systems. Identifying individuals through facial features helps reduce dependence on traditional methods such as passwords, access cards, or paper-based attendance systems.

Nowadays, many intelligent systems have integrated face recognition technology to increase processing speed and improve user experience. In addition, this technology effectively supports management, surveillance, and information authentication in various fields.

In education and research, face recognition is considered one of the most typical applications of artificial intelligence and image processing, helping students approach modern technologies and improve practical programming skills.

1.3 Research Subjects and Research Scope

1.3.1 Research Subjects

- The Java programming language and the OpenCV library used to develop the face recognition application.
- Image processing and face detection techniques in OpenCV, combined with face recognition models to identify users through camera input.
- Methods of developing desktop applications using Java Swing and organizing the program based on object-oriented programming (OOP).
- Database connectivity using JDBC for storing user information and face recognition history.

1.3.2 Research Scope

- The application is developed as a Desktop Application running on the Windows operating system.
- The system uses a computer webcam to capture images and perform real-time face recognition.
- A MySQL database is used to store user information and recognition data.
- The system mainly focuses on basic face detection and recognition under relatively stable lighting conditions and frontal face angles.
- The project does not cover more complex cases such as face recognition with masks, partially occluded faces, or extremely poor lighting conditions.

1.4 Technologies Used

The project uses a combination of programming, user interface, image processing, and database technologies. Java is used as the main programming language because it supports object-oriented programming and desktop application development. Java Swing is used to build the graphical interface, including the main window, functional tabs, buttons, labels, preview area, and result table [1], [2].

OpenCV is used for computer vision processing. In this project, OpenCV provides access to the webcam, image frame processing, face detection with YuNet, and face recognition with SFace. JDBC is used to connect the Java

application to the local MySQL/MariaDB database. XAMPP and phpMyAdmin support database creation and management during development [3], [4], [5], [6], [8], [9], [10].

Table 1.1. Technologies and Their Roles in the Face Recognition Application

Technology	Role
Java Language	**Main programming language**
Java Swing	**Desktop GUI development**
OpenCV	**Image processing and face recognition**
JDBC	**Database connectivity**
XAMPP	**Local MySQL server environment**
MySQL	**System data storage**
phpMyAdmin	**Web-based database management tool**

1.5 System Architecture

The system is designed with a simple layered architecture. The user interface layer receives user actions and displays camera images, registration results, recognition results, and gallery data. The AI processing layer performs face detection and feature extraction using OpenCV models. The data layer stores face embeddings in gallery.dat and stores recognition logs in the database.

The general workflow of the system starts when the application loads OpenCV and opens the main window. In the registration function, the user captures a face image, the system detects the face, extracts the embedding, and saves it with the user's name. In the recognition function, the system continuously captures frames from the webcam, detects the face, extracts the current embedding, compares it with stored embeddings, and displays the final result on the interface.

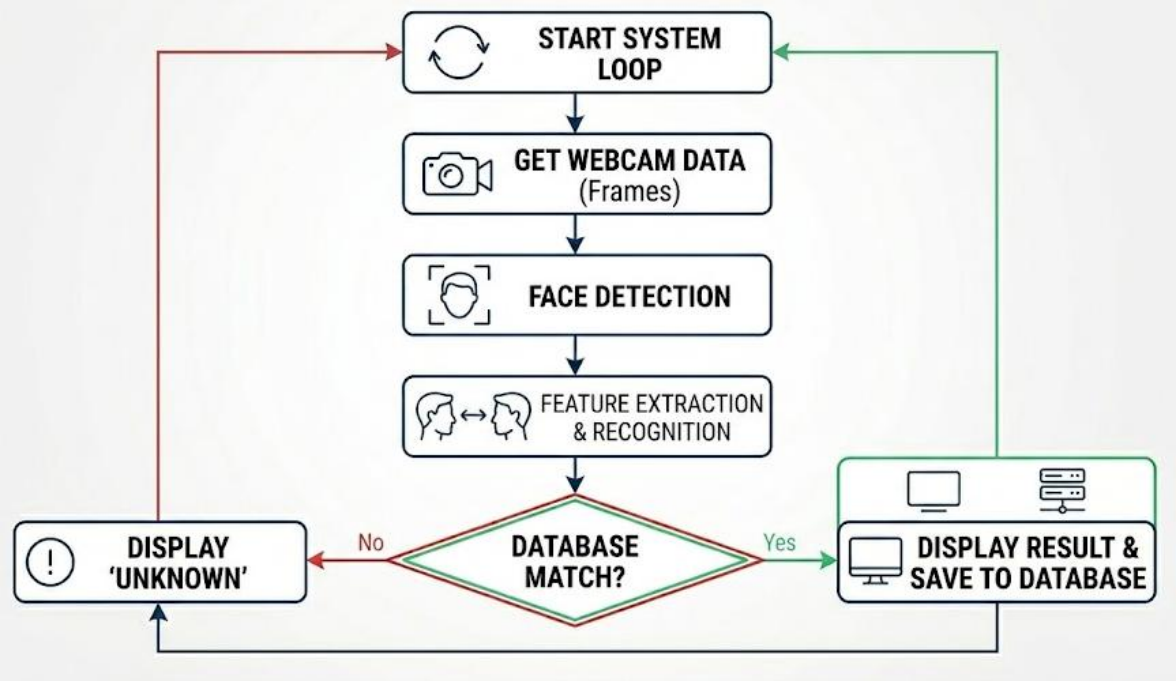


Figure 1.1. Face Recognition System Workflow

CHAPTER 2: THEORETICAL BASIS

2.1 Java Programming Language

Java is an object-oriented programming language originally developed by Sun Microsystems and currently owned by Oracle. Java was designed to be simple, secure, and platform-independent through the Java Virtual Machine (JVM) [1].

One of Java's most important features is the principle "Write Once, Run Anywhere", which allows programs to run on different operating systems without modifying the source code. In addition, Java strongly supports Object-Oriented Programming (OOP), making software easier to organize, maintain, and expand [1].

In this project, Java is used as the main programming language to develop the entire face recognition system. Functions such as user interface design, data processing, database connectivity, and system workflow management are all implemented using Java.

2.2 Java Swing

Java Swing is a graphical user interface (GUI) library provided in Java. Swing supports many interface components such as windows, buttons, labels, panels, and input fields, allowing developers to build interactive desktop applications [2].

Swing operates based on the Event-Driven Programming model, enabling applications to respond to user actions such as button clicks, menu selections, and mouse interactions. In addition, Swing supports flexible interface customization and layout organization through Layout Managers [2].

In this project, Java Swing is used to build the main interface of the face recognition system. Components such as the main window, webcam display area, functional menus, and control buttons are all developed using the Swing library.

2.3 Java Database Connectivity

JDBC (Java Database Connectivity) is a Java API that allows applications to connect to and work with databases. JDBC supports operations such as querying, inserting, updating, and deleting data using SQL statements [3].

JDBC acts as an intermediary between Java applications and database management systems. Through JDBC, programs can establish database connections, send SQL commands, and receive returned results from the database system [3].

Some important components of JDBC include:

- DriverManager
- Connection
- Statement
- ResultSet

In this project, JDBC is used to connect the Java application to the MySQL/MariaDB database running through XAMPP. JDBC supports storing user information and face recognition history of the system.

2.4 Open Source Computer Vision Library

OpenCV (Open Source Computer Vision Library) is an open-source library specialized in image processing and computer vision. The library provides many tools for image analysis, video processing, and real-time object recognition [4].

OpenCV supports many important functions such as image processing, face detection, object recognition, webcam video processing, and artificial intelligence applications in computer vision. In addition, OpenCV can be integrated with multiple programming languages such as C++, Python, and Java [4].

In this project, OpenCV is used to process images captured from the webcam, detect faces, and support the real-time face recognition process. The OpenCV library plays an important role in the entire image processing workflow of the system.

2.5 Face Detection

Face Detection is the process of identifying the location of human faces in images or videos. The system analyzes image data captured from the camera to locate areas containing user faces before performing the recognition process [4], [5].

In modern image processing systems, Face Detection is an important step that reduces unnecessary data processing and improves the accuracy of face recognition. The detection result is usually represented as a bounding box surrounding the detected face.

In this project, the system uses the YuNet model provided by OpenCV to perform real-time face detection from webcam input. After detecting a face, the data is transferred to the face recognition stage to identify the user [5].

2.6 Face Recognition

Face Recognition is the process of identifying or verifying a person's identity based on facial features. After a face is detected, the system analyzes and extracts important facial characteristics to compare them with previously stored data [6].

In modern systems, Face Recognition commonly uses artificial intelligence models to generate embedding vectors for each face. Comparing these vectors allows the system to determine the similarity between different faces [6].

In this project, the system uses the SFace model provided by OpenCV to perform face recognition. After detecting a face using YuNet, the SFace model extracts facial features and compares them with registered data to identify the user.

2.7 YuNet and SFace Models

YuNet and SFace are two face-processing models integrated into the OpenCV library to support real-time face detection and recognition [5], [6], [7].

YuNet is a model used for detecting faces in images or videos. This model can identify face locations with high processing speed and is suitable for real-time systems using webcam input [5], [7]. After detecting a face, the system creates a facial region for the next recognition stage.

SFace is a face recognition model used to extract facial features in the form of embedding vectors. These vectors are compared with previously stored data to determine the identity of users.

In this project, YuNet is used for the face detection process, while SFace is used for face recognition and comparison with registered facial data in the system. The combination of these two models enables the system to perform real-time face recognition with appropriate accuracy and processing speed.

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

3.1 System Functional Analysis

The system functions are designed around three main user needs: registering a new face, recognizing a face in real time, and managing stored face data. The registration function prepares data for the recognition stage. The recognition function uses the registered data to identify people from live camera input. The management function allows users to check and update the local face gallery.

The application also includes supporting functions such as opening and closing the webcam, displaying status messages, drawing face bounding boxes, validating user input, saving logs, and handling errors. These supporting functions are important because they make the program more stable and easier to use.

The face recognition system is developed as a desktop application that supports real-time face detection and recognition through webcam input. The system includes the following main functions:

Table 3.1. Main Functions of the Face Recognition System

Function	Description
Webcam Access	Opens the webcam and displays real-time video on the application interface.
Face Detection	Detects face locations in camera frames using OpenCV and the YuNet model.
Face Recognition	Extracts facial features and compares them with stored data using the SFace model.
Face Registration	Allows users to register new faces into the system for later recognition.
Data Storage	Stores registered face data and recognition history using local files and MySQL/MariaDB.
Database Connection	Connects the Java application to the database through JDBC.

User Interface	Provides an easy-to-use desktop interface built with Java Swing.

These functions work together to allow the system to capture images, process faces, recognize users, and store recognition results efficiently.

3.2 Database Design

The database design of the face recognition application is kept simple, clear, and suitable for a Java desktop project. The main recognition data is divided into two parts. Face embedding data is stored in the local file gallery.dat, while the recognition history is stored in a MySQL database [3], [8]. This design helps the system separate AI data from log data and makes each part easier to manage.

The system does not store original face images in the database. Instead, the application saves only the recognition log, including the name of the detected person, the recognition time, and the similarity score. This approach reduces database size, protects image data better, and still provides enough information for checking recognition history during testing.

The database used in the project is named detection_log. It is created in MySQL/MariaDB through XAMPP. The schema.sql file creates the database with UTF-8 support using utf8mb4 and creates a table with the same name detection_log [8], [9]. The table is designed to store each recognition event as one record.

The detection_log table contains four main fields. The id field is an integer primary key and increases automatically. The person_name field stores the recognized person name or UNKNOWN when the system cannot identify the face. The timestamp field stores the time when the recognition event happens and can use the current timestamp as the default value. The score field stores the confidence score as a double value from 0.0 to 1.0.

This database structure is suitable for the current scope of the project because it supports basic requirements such as saving a new log, displaying all logs, and filtering logs by person name. In the future, the database can be expanded with additional tables such as users, departments, classes, attendance

sessions, or devices if the application is developed into a real attendance management system.

Table 3.2. Structure of the detection_log Table

Field	Data Type	Description
id	INT	Primary key, automatically increases for each record
person_name	VARCHAR(255)	Name of the recognized person
timestamp	DATETIME	Time when the face was recognized
score	DOUBLE	Recognition confidence score

3.3 Class Design

The class structure follows the object-oriented programming approach. Each class has a clear responsibility, which helps reduce confusion when reading or maintaining the program. For example, MainFrame manages the overall window, CameraPanel handles camera display, FaceDetector handles YuNet detection, and FaceRecognizer handles SFace feature extraction.

This separation also helps the group test each module more easily. The AI pipeline can be tested separately from the user interface, while gallery management and database logging can be checked independently. As a result, errors can be found and fixed faster during development.

The system is organized into several main classes, each responsible for a specific function in the application. This class-based structure helps separate the user interface, face processing, data storage, and database connection.

Table 3.3. Main Classes of the Face Recognition Application

Class	Description
Main	Starts the application and loads the OpenCV library.
MainFrame	Creates the main application window and manages different panels.
CameraPanel	Displays real-time webcam images.

RegisterPanel	Handles new face registration.
RecognizePanel	Handles face recognition from webcam input.
ManagePanel	Manages registered face data.
FaceDetector	Detects faces using the YuNet model.
FaceRecognizer	Extracts and compares facial features using the SFace model.
GalleryEntry	Represents stored face information.
GalleryRepository	Manages registered face data in the gallery.
EmbeddingSerializer	Saves and loads face embedding data.
DatabaseConnection	Connects the application to the database.
DetectionLogDAO	Stores and retrieves face recognition history.

This class design makes the program easier to maintain, understand, and expand in the future.

3.4 User Interface Design

The interface is organized into three tabs to match the main workflow of the system. The Register tab is used to add new users, the Recognize tab is used to identify faces in real time, and the Manage tab is used to view or remove registered data. This tab-based design is simple but effective for a student project because users can understand the application quickly.

The interface also uses colors and status labels to provide feedback. Successful actions are displayed with positive messages, while errors such as missing name, no captured image, or no detected face are shown clearly. This design reduces user mistakes and improves the overall interaction experience.

The user interface of the system is developed using Java Swing to provide a simple, intuitive, and easy-to-use desktop environment. The interface allows users to interact with the webcam, register faces, recognize users, and manage stored face data.

The main interface of the application includes several functional panels such as webcam display, face registration, face recognition, and face data

management. Buttons, menus, and panels are organized to support convenient operation and clear interaction for users.

During operation, the webcam image is displayed in real time on the interface, allowing users to monitor the face detection and recognition process directly. Recognition results and system messages are also displayed visually to improve user experience.

CHAPTER 4: IMPLEMENTATION

4.1 Application Structure

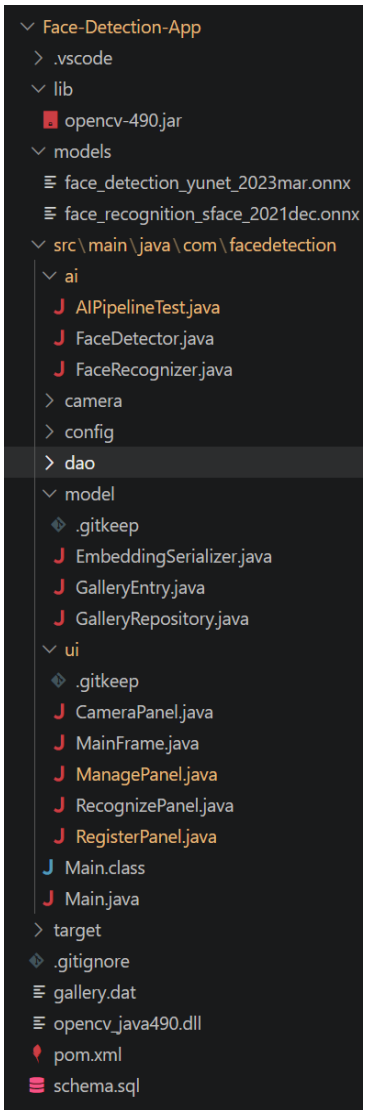


Figure 4.1. Application Project Structur

The application is organized into multiple packages to separate the user interface, face processing, and database management modules. This structure helps improve source code organization, system maintenance, and future expansion of the project.

The project follows a modular architecture where each package has a specific responsibility. The ai package contains the main artificial intelligence modules, including face detection and face recognition using the YuNet and SFace models provided by OpenCV. These classes process webcam frames, detect faces, extract facial embeddings, and compare similarity scores.

The camera package is responsible for webcam processing and real-time image capture. This module continuously receives image frames from the webcam and transfers them to the face detection and recognition pipeline.

The dao package manages database operations through JDBC. It includes classes for database connection handling and recognition log management. The system stores recognition history in a local MySQL/MariaDB database.

The model package contains data models used throughout the application. These classes represent gallery entries, embedding data, and serialization objects used for saving and loading face information.

The ui package contains Java Swing graphical user interface components. This package manages the main application window, registration panel, recognition panel, and gallery management panel.

In addition to the source code packages, the project also includes several important resources and configuration files:

- models/: stores the YuNet and SFace ONNX AI models
- gallery.dat: stores serialized face embedding data
- schema.sql: database initialization script
- opencv_java490.dll: OpenCV native library for Java
- pom.xml: Maven dependency configuration file

This package organization makes the application easier to develop, debug, test, and maintain. It also supports better separation between the graphical interface, AI processing modules, and database management components.

4.2 MainFrame and User Interface

The main interface of the application is developed using Java Swing. The MainFrame class acts as the central window of the face recognition system and is responsible for creating the graphical user interface, managing functional panels, and controlling webcam operations between tabs [2].

The MainFrame class extends JFrame, allowing the application to operate as a desktop window. The constructor initializes the application title, window size, screen position, and default close operation.

```
setTitle(title: "Face Detection App – BTL Java");
setSize(width: 1100, height: 700);
setLocationRelativeTo(c: null);
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

Figure 4.2. MainFrame Window Initialization Code

This code sets the title, size, screen position, and close operation of the main application window. The interface size is configured to provide enough space for webcam display, control buttons, and recognition results.

The application initializes three main functional panels:

- RegisterPanel
- RecognizePanel
- ManagePanel

```
// Khởi tạo các panel (camera CHƯA mở)
registerPanel   = new RegisterPanel();
recognizePanel = new RecognizePanel();
managePanel     = new ManagePanel();
```

Figure 4.3. Functional Panel Initialization Code

These panels separate the major functions of the application into independent modules. This modular structure makes the system easier to maintain, debug, and expand in future development.

The graphical interface is organized using JTabbedPane.

```
// TabbedPane
tabbedPane = new JTabbedPane();
tabbedPane.setFont(new Font(name: "Arial", Font.BOLD, size: 14));
tabbedPane.addTab(title: " Đăng ký",    registerPanel);
tabbedPane.addTab(title: "Nhận diện", recognizePanel);
tabbedPane.addTab(title: "Quản lý",    managePanel);
```

Figure 4.4. TabbedPane Interface Organization Code

The tabbed interface allows users to switch between different functions conveniently while keeping the interface clean and organized.

The Register tab is used to capture and save new face data into the local gallery. Users can enter a name, capture an image from the webcam, and register a new face into the recognition system.

The Recognize tab performs real-time face recognition using webcam input. The system continuously detects faces, extracts facial features, compares them with stored embeddings, and displays recognition results on the interface.

The Manage tab is responsible for gallery management. Users can view registered face data, refresh the gallery table, delete selected entries, or clear all registered data from the local storage.

The application uses the event-driven programming model of Java Swing. User actions such as changing tabs, clicking buttons, or closing the application automatically trigger corresponding processing events inside the system [2].

To manage webcam resources efficiently, the system controls camera activation dynamically when users switch between tabs.

```
// Quản lý camera theo tab: chỉ mở camera của tab đang chọn
tabbedPane.addChangeListener(e -> {
    int newTab = tabbedPane.getSelectedIndex();
    if (newTab == currentTab) return;

    // Dừng camera của tab cũ
    stopCamera(currentTab);

    // Mở camera của tab mới
    startCamera(newTab);

    currentTab = newTab;
});
```

Figure 4.5. Camera Control When Switching Tabs

When a tab change occurs, the application stops the webcam of the previous tab and activates the webcam of the selected tab. This mechanism helps avoid webcam conflicts and reduces unnecessary CPU usage during real-time image processing.

The application automatically starts the webcam for the Register tab when the program launches.

```
// Mở camera mặc định cho Tab 0 (Đăng ký)
registerPanel.startCamera();
```

Figure 4.6. Default Camera Startup for the Register Tab

This allows users to begin face registration immediately after opening the application.

The system also handles proper webcam shutdown when the application closes.

```
// Đóng app: dừng tất cả camera
addWindowListener(new WindowAdapter() {
    public void windowClosing(WindowEvent e) {
        registerPanel.stopCamera();
        recognizePanel.stopCamera();
    }
});
```

Figure 4.7. Webcam Resource Release When Closing the Application

This code ensures that all active webcam processes are stopped correctly before the program terminates. Proper resource cleanup prevents the webcam from remaining locked after application shutdown.

The interface layout is designed to separate webcam display areas, control buttons, recognition results, and status information clearly. This improves usability and helps users monitor the face detection and recognition process more effectively.

Overall, the MainFrame class and Java Swing interface provide the central interaction layer of the application. They coordinate communication between the user, webcam system, AI processing modules, and database components throughout the entire face recognition workflow.

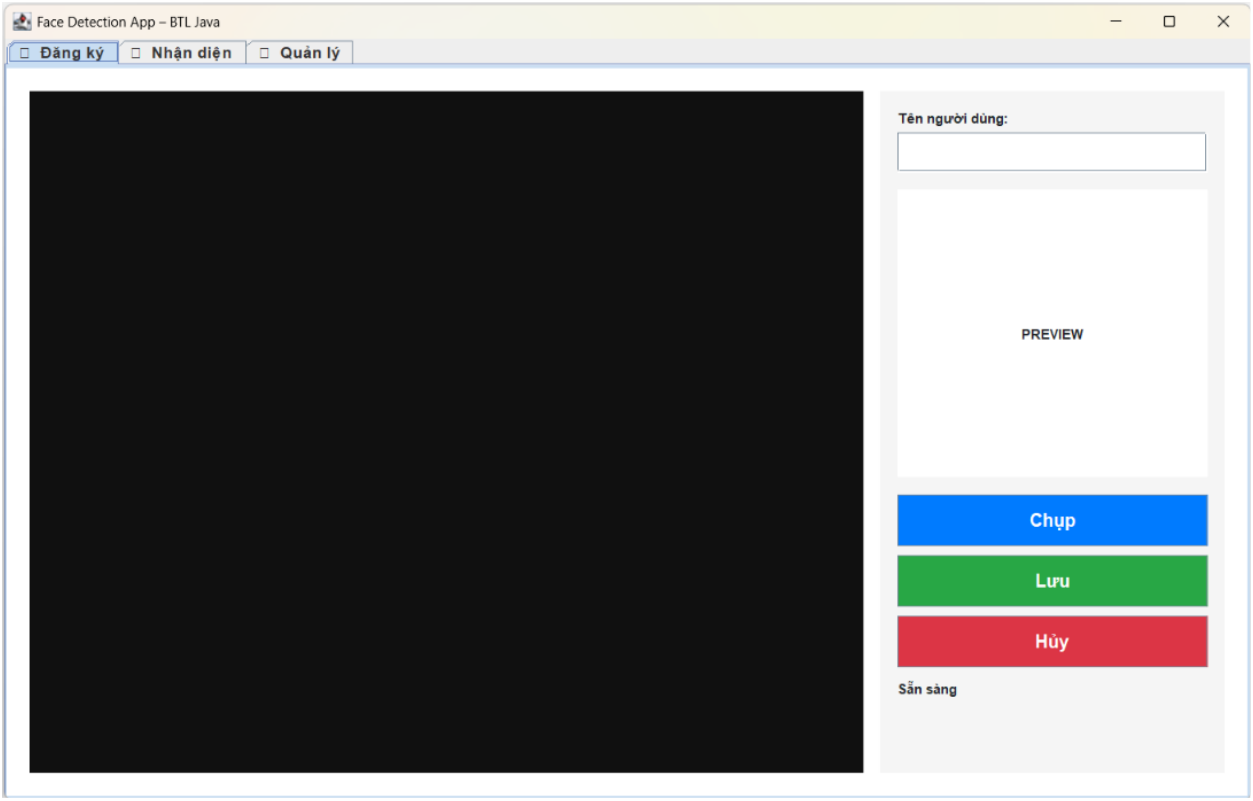


Figure 4.8. Main Interface of the Face Recognition Application

4.3 Webcam Processing and Camera Display

The webcam processing function is implemented mainly in the CameraPanel class. This class is responsible for accessing the computer webcam, capturing real-time image frames, converting OpenCV images into Java images, and displaying them on the Swing interface [4].

The system uses OpenCV's VideoCapture class to connect to the default webcam of the computer [4].

```
camera = new VideoCapture(index: 0);
frame  = new Mat();
```

Figure 4.9.Webcam and Frame Initialization Code

This code initializes the default webcam and creates a Mat object to store image frames captured from the camera. The value 0 represents the default camera device of the computer. The frame variable is used to hold each image frame before it is processed and displayed on the Swing interface.

```
if (!camera.isOpened()) {
    JOptionPane.showMessageDialog(this, message: "Không mở được webcam");
    return;
}

running = true;
```

Figure 4.10. Webcam Opening Validation Code

This code checks whether the webcam is opened successfully. If the camera cannot be accessed, the system displays an error message and stops the webcam initialization process using the return statement.

If the webcam is opened successfully, the variable running is set to true, indicating that the webcam processing loop is active and ready to capture image frames continuously.

```
    Thread t = new Thread(() -> {
        while (running) {
            if (camera.read(frame)) {

                FaceDetector detector = faceDetector;
                if (detector != null && !frame.empty()) {
                    try {
                        Mat faces = detector.detect(frame);
                        if (faces != null && faces.rows() > 0) {
                            detector.drawFaces(frame, faces);
                        }
                    } catch (Exception e) {
                        // Bỏ qua lỗi detect để không crash camera thread
                    }
                }

                image = matToImage(frame);
                repaint();
            }

            try {
                Thread.sleep(millis: 30);
            } catch (Exception e) {
                e.printStackTrace();
            }
        }
    });

    t.setDaemon(on: true);
    t.start();
}
```

Figure 4.11. Real-time Webcam Processing Thread

This code creates a background thread for real-time webcam processing. The loop continuously captures image frames from the webcam while the running variable remains true.

The camera.read(frame) function reads the current image frame from the webcam and stores it inside the frame object. After processing each frame, the repaint() method updates the Swing interface to display the newest webcam image.

The Thread.sleep(30) function temporarily pauses the thread for 30 milliseconds to control the frame update speed and reduce unnecessary CPU usage during continuous webcam processing.

```
protected void paintComponent(Graphics g) {
    super.paintComponent(g);

    if (image != null) {
        g.drawImage(image, getWidth(), y: 0, -getWidth(), getHeight(), observer: null);
    } else {
        g.setColor(Color.WHITE);
        g.drawString(str: "Đang mở webcam...", getWidth() / 2 - 60, getHeight() / 2);
    }
}
```

Figure 4.12. Webcam Image Rendering Code

This method is responsible for drawing the webcam image on the Swing panel. The paintComponent() method is automatically called whenever the interface needs to be updated.

If the image object is not null, the latest webcam frame is drawn on the panel using drawImage(). The image is scaled to fit the panel size. The negative width value is used to flip the webcam image horizontally, creating a mirror-like camera view.

If the webcam image is not available yet, the system displays the message "Đang mở webcam..." to inform users that the camera is being initialized.

```java
public void stopCamera() {
    running = false;
    faceDetector = null;
    if (camera != null) {
        camera.release();
    }
}
```

Figure 4.13. Webcam Stop and Resource Release Code

This method stops the webcam processing and releases the camera resource. The running variable is set to false to stop the camera loop. After that, camera.release() frees the webcam device, preventing camera conflicts when switching tabs or closing the application.

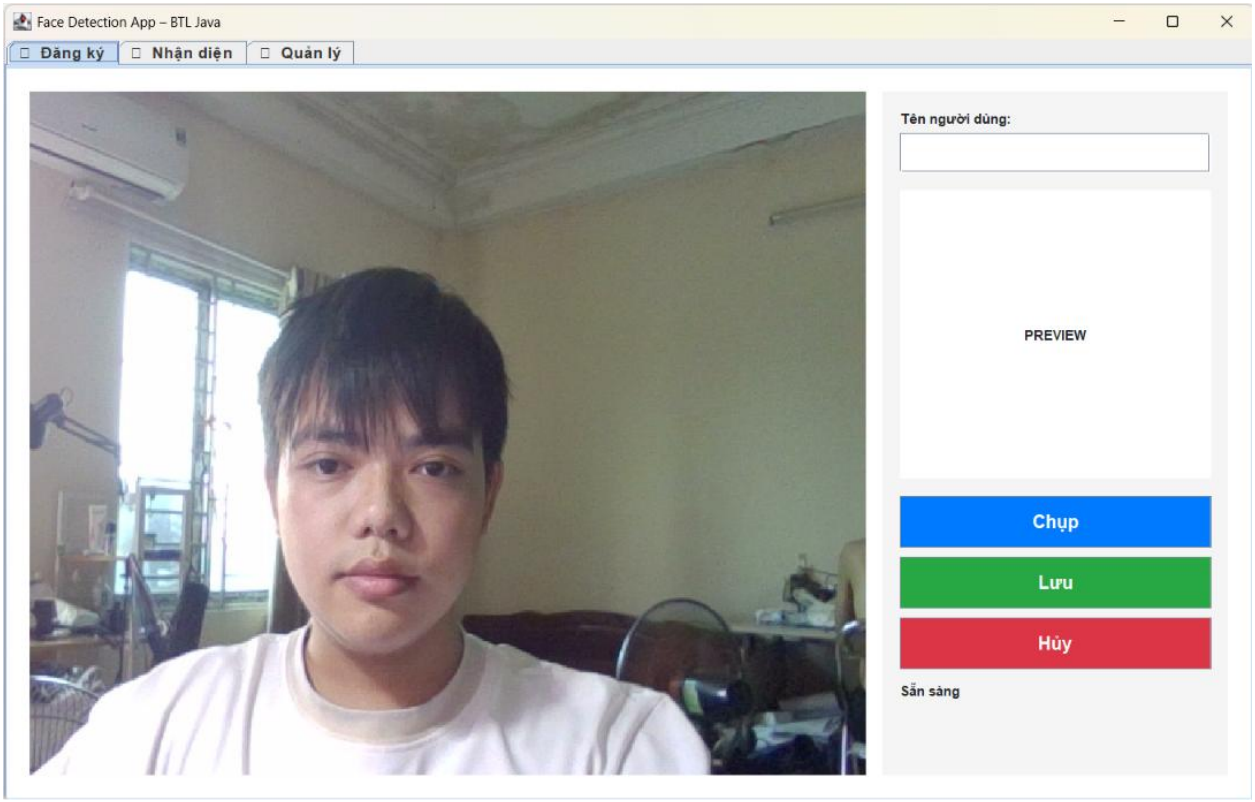


Figure 4.14. User Interface During Face Registration

4.4 Face Detection Implementation

The face detection function is implemented mainly in the FaceDetector class. This class uses the FaceDetectorYN module of OpenCV and loads the model file face_detection_yunet_2023mar.onnx from the models folder. YuNet

is selected because it is lightweight, fast, and suitable for detecting faces from webcam frames in real time [5], [7].

When a frame is received from the webcam, the system creates an empty Mat object to store the detection result. Before detection, the input size of the detector is updated according to the current frame size. This step is important because webcam frames may have different widths and heights depending on the camera device. After that, the detect() method analyzes the frame and returns a matrix containing detected face information [5].

Each detected face is represented by a row in the result matrix. The first values in each row include the x coordinate, y coordinate, width, and height of the bounding box. These values are used to draw a rectangle around the face on the camera screen. The drawFaces() method uses Imgproc.rectangle() to display the bounding box, helping users clearly see whether the system has detected the face correctly.

In the CameraPanel class, the FaceDetector object can be attached or removed through the setFaceDetector() method. When recognition mode is active, CameraPanel calls the detector continuously and draws the face boxes on the live video. When recognition is stopped, the detector is removed to reduce unnecessary processing. The detection process is also placed inside a try-catch block so that small detection errors do not stop the webcam thread.

Overall, the YuNet and OpenCV detection module acts as the first step of the whole AI pipeline. It locates the face area from the webcam image and sends this information to the recognition stage. If no face is detected, the system does not continue feature extraction, which helps avoid wrong recognition results and improves processing efficiency.

4.5 Face Recognition Implementation

Face recognition is implemented mainly through the FaceRecognizer class and the recognition logic inside RecognizePanel. The system uses the OpenCV FaceRecognizerSF module with the model file face_recognition_sface_2021dec.onnx [6], [7]. While YuNet is responsible for finding the face location, SFace is responsible for extracting the facial features used for comparison.

After YuNet detects a face, the SFace model first aligns and crops the face using the alignCrop() method. This step helps normalize the face area before feature extraction. Then the feature() method converts the aligned face into an

embedding vector [6]. In this project, the embedding is converted from Mat to a float array so it can be saved and compared more easily in Java.

The recognition process compares the embedding of the current face with all embeddings stored in gallery.dat. For each registered person, the system calculates cosine similarity between the current embedding and the stored embedding. Cosine similarity is suitable for this task because it measures the direction similarity between two feature vectors instead of comparing raw image pixels [6].

The system uses a threshold value of 0.363. If the best similarity score is lower than this threshold, the result is shown as UNKNOWN. If the score is equal to or higher than the threshold, the system returns the name of the person with the highest score. The result is displayed on the interface together with the confidence percentage, making it easier for users to understand the recognition result.

The recognition loop in RecognizePanel runs on a background thread and processes the camera image every 400 milliseconds. This design keeps the interface responsive while still allowing real-time recognition. The UI is updated through SwingUtilities.invokeLater(), which ensures that Swing components are updated safely from the event dispatch thread.

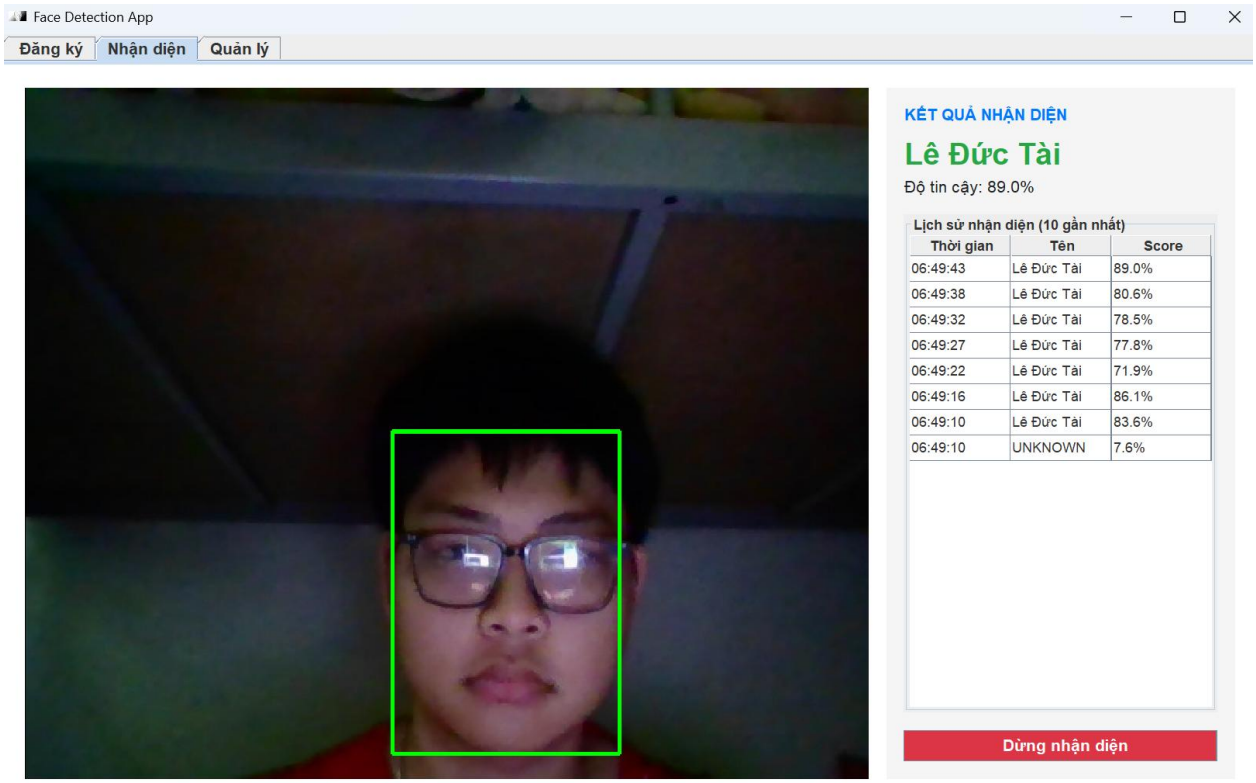


Figure 4.15. Real-time Face Recognition Interface

4.6 Face Registration and Gallery Management

The registration function allows users to add a new face to the system. This function is implemented in RegisterPanel. The user first enters a name, then captures an image from the webcam. The captured image is displayed in the preview area so the user can check it before saving.

When the user clicks the save button, the system validates the input data. It checks whether the name field is empty, whether an image has been captured, and whether at least one face has been detected in the captured image. These validation steps help prevent invalid data from being saved into the gallery.

After validation, the system uses the SFace model to extract the embedding vector from the detected face. The embedding represents the important facial features of the user. Instead of storing the original image, the system stores the name and the embedding vector. This method reduces storage size and makes the recognition process faster because the system only needs to compare numerical vectors.

The GalleryEntry class is used to represent one registered person. It contains the person's name and the embedding vector. The GalleryRepository class manages a list of GalleryEntry objects. When a new face is registered, GalleryRepository adds a new entry to the list and saves the updated list into gallery.dat.

The EmbeddingSerializer class is responsible for saving and loading gallery data using ObjectOutputStream and ObjectInputStream. When the program starts, the existing gallery.dat file is loaded into memory. When a new person is added or the gallery is cleared, the file is updated. This makes the data persistent, so registered faces can still be recognized after the application is closed and opened again.

Gallery management is implemented in ManagePanel. This panel displays the list of registered users in a table, including the order number, user name, and embedding dimension. Users can refresh the table, delete one selected person, or clear the whole gallery. These functions make the application easier to maintain during testing and normal use.

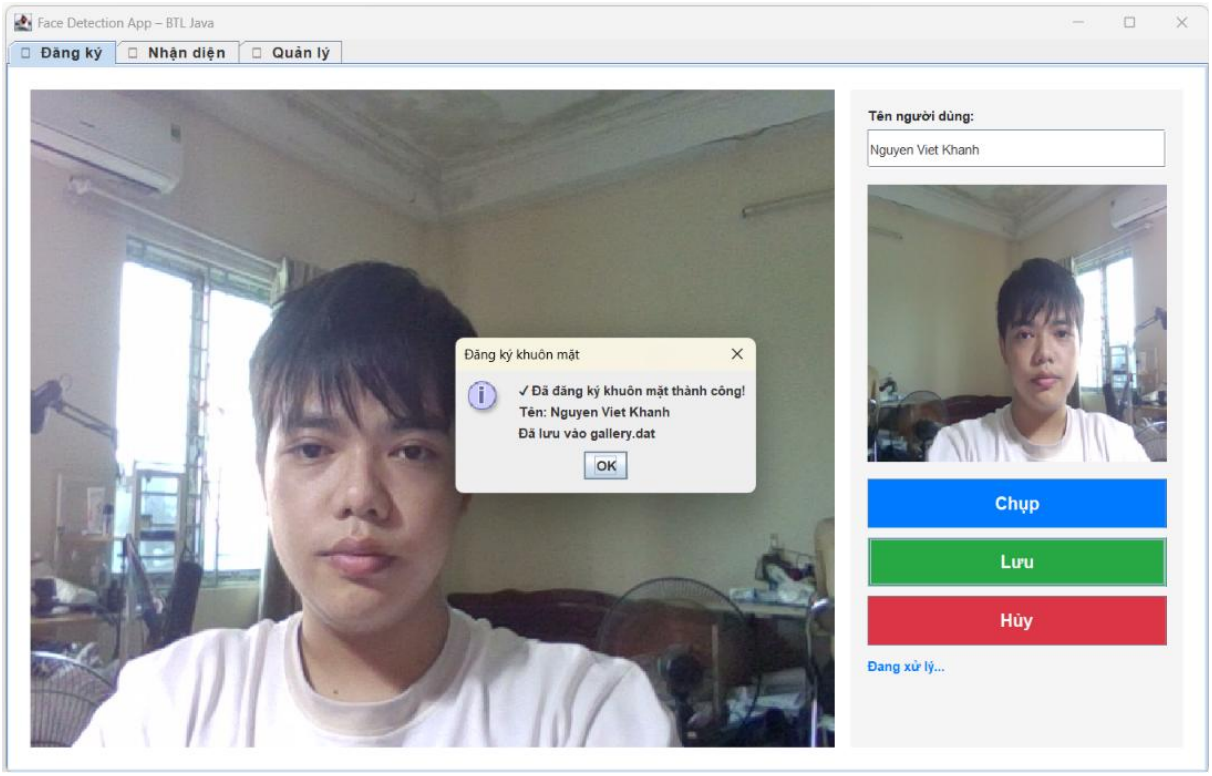


Figure 4.16. Face Registration Interface

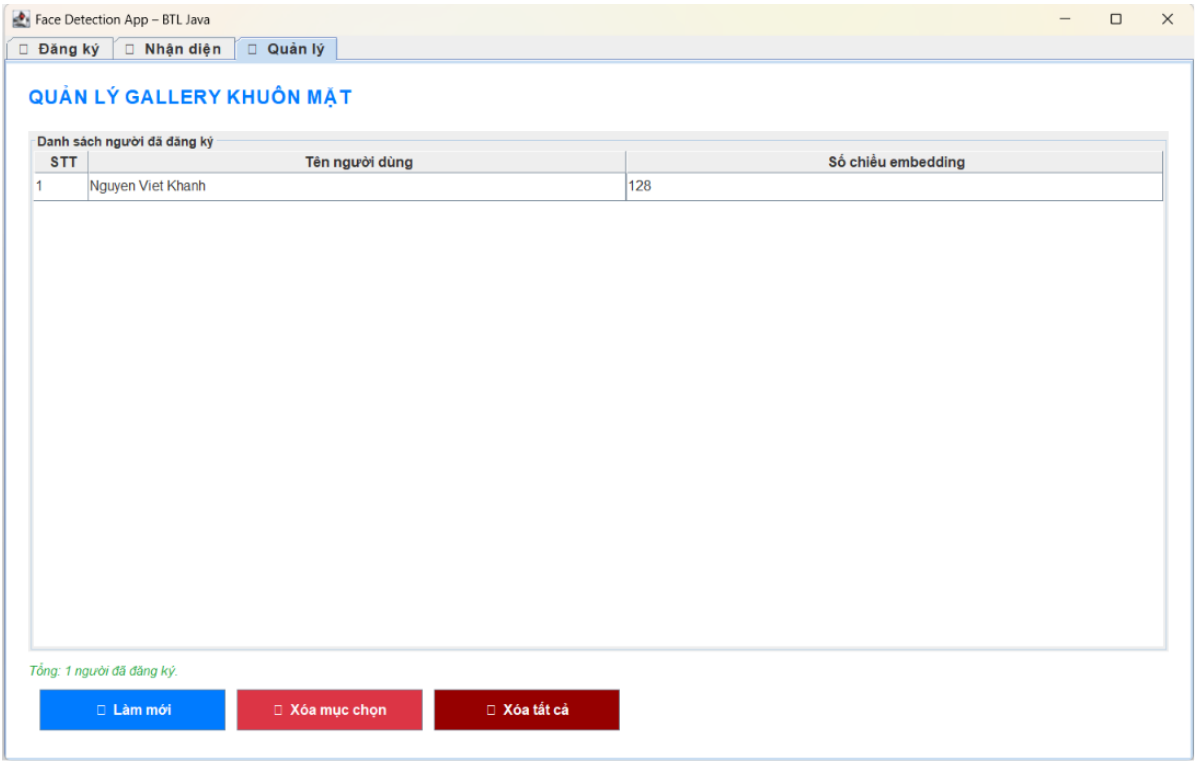


Figure 4.17. Gallery Management Interface

4.7 Database Connection and Recognition Log

Besides storing registered face embeddings in gallery.dat, the application also uses a MySQL/MariaDB database to save recognition history. The database part is implemented through JDBC and is separated into two main classes: DatabaseConnection and DetectionLogDAO [3], [8]. This separation makes the source code cleaner because the user interface does not directly execute SQL statements.

The DatabaseConnection class is responsible for creating and managing the connection between the Java application and MySQL. In this project, the database host is localhost, the port is 3306, the database name is detection_log, the username is root, and the password is empty because the project runs on a local XAMPP environment. The JDBC URL also includes useSSL=false, allowPublicKeyRetrieval=true, serverTimezone=Asia/Ho_Chi_Minh, and characterEncoding=UTF-8 to make the connection more stable and suitable for Vietnamese text.

DatabaseConnection is designed using the Singleton pattern. The getInstance() method ensures that the program uses only one shared DatabaseConnection object. The getConnection() method checks whether the current connection is null, closed, or invalid. If the connection is not available, the method creates a new connection using DriverManager.getConnection() [3]. This helps the program avoid opening many unnecessary database connections during execution.

After connecting successfully, the program calls createTableIfNotExists(). This method executes a CREATE TABLE IF NOT EXISTS statement for the detection_log table. As a result, even if the table has not been created manually, the application can still prepare the required table automatically. The table contains id, person_name, timestamp, and score. The id column is the primary key with AUTO_INCREMENT, timestamp uses DATETIME with CURRENT_TIMESTAMP as the default value, and score is stored as DOUBLE.

The schema.sql file provides a more complete database setup script for manual installation through phpMyAdmin or MySQL command line. It creates the detection_log database with utf8mb4 character set and utf8mb4_unicode_ci

collation, selects the database using USE detection_log, creates the detection_log table with InnoDB engine, and inserts several sample records for testing. Therefore, before running the project, users can either run schema.sql manually or let the Java application create the table automatically after the database already exists.

The DetectionLogDAO class is the Data Access Object layer of the system. It contains methods for inserting and reading recognition logs. The insertLog(String personName, Timestamp timestamp, double score) method uses PreparedStatement to insert data into the detection_log table. PreparedStatement is safer and more organized than building SQL strings directly because the values are passed through parameters [3]. The class also has an overloaded insertLog(String personName, double score) method that automatically uses the current system time.

For reading data, DetectionLogDAO provides getAllLogs() and getLogsByPerson(String personName). The getAllLogs() method selects id, person_name, timestamp, and score from the detection_log table and sorts the result by timestamp in descending order. This means the newest recognition records appear first. The getLogsByPerson() method uses a WHERE condition to return only logs of a specific person. These methods can be used later to build a log management screen or an attendance report function.

The database log is connected to both the registration and recognition processes. In RegisterPanel, after a face is captured, detected, converted into an embedding, and saved into gallery.dat, the system calls dao.insertLog(name, score) with score equal to 1.0 to record the registration event. In RecognizePanel, the system records recognition results during real-time recognition. The displayed score is shown as a percentage on the interface, but before saving to the database it is converted back to the range 0.0 to 1.0.

To avoid creating too many repeated records, RecognizePanel uses a log interval of 5000 milliseconds. If the same person is continuously recognized, the system does not save a new log every frame. It only saves again after the time interval has passed or when the recognized name changes. This design keeps the

database history easier to read and prevents unnecessary database overload while the webcam is running.

If the database connection fails, the DAO methods catch SQLException and return false instead of stopping the whole application. This is important because the main face registration and recognition data is still stored in gallery.dat. However, the recognition history will not be saved until MySQL is started and the detection_log database is available. Therefore, when testing this part, the user should start XAMPP, turn on MySQL, create the detection_log database, and then run the Java application [9], [10].

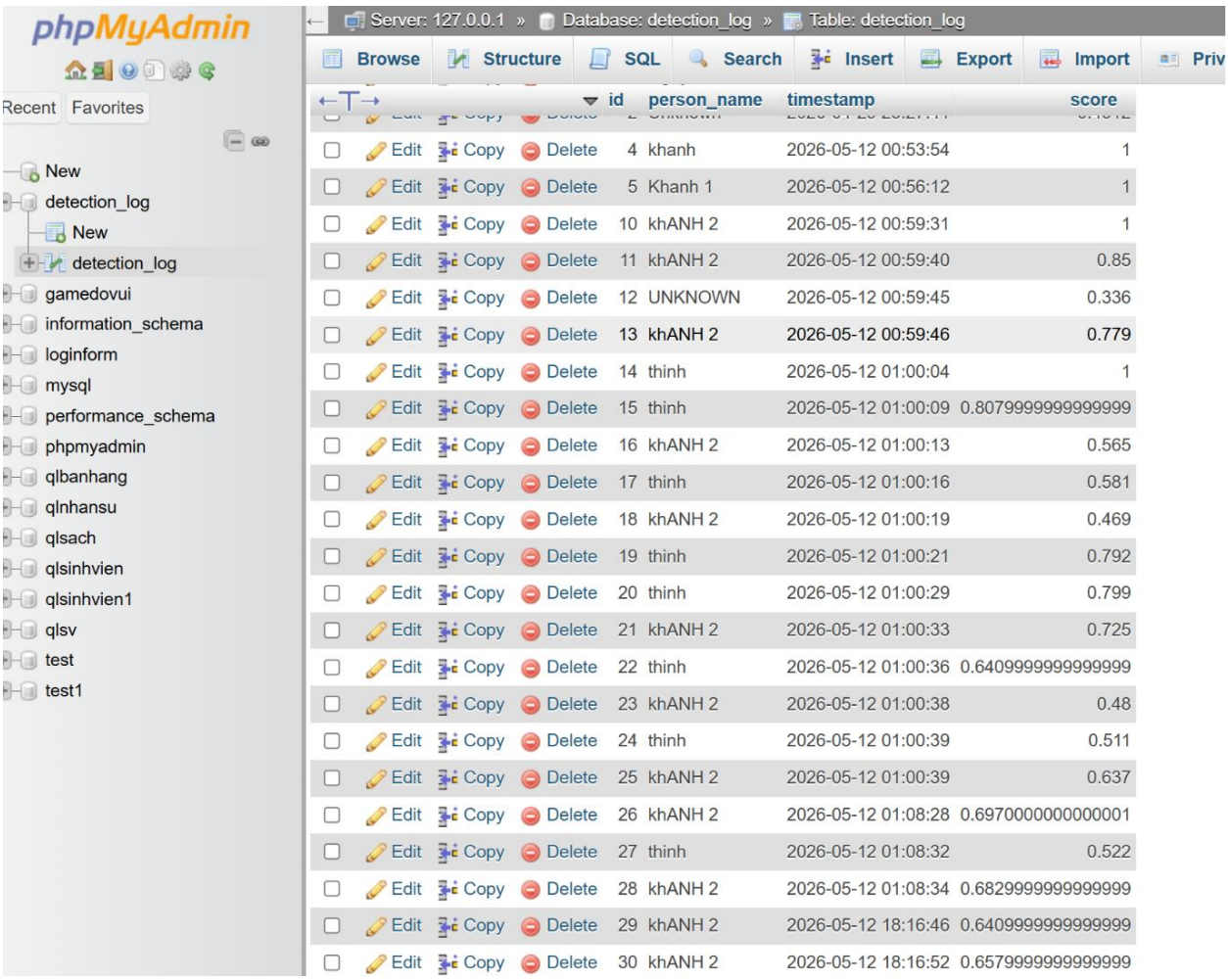


Figure 4.18. Detection Log Table in phpMyAdmin

4.8 Experimental Results

After completing the implementation, the system was tested on a Windows computer with XAMPP, MySQL, OpenCV 4.9.0, a webcam, and the two ONNX models in the models folder. The testing process focused on checking the main

workflow of the application: starting the program, opening the webcam, detecting faces, registering faces, recognizing registered users, managing gallery data, and saving recognition logs into the database.

The first test was the startup test. The Main class loads opencv_java490.dll from the project root folder and prints the OpenCV version to the console. After that, SwingUtilities.invokeLater() opens MainFrame. The application window appears with three main tabs: Register, Recognize, and Manage. This confirms that the OpenCV native library and Java Swing interface are loaded successfully.

The second test was the AI pipeline test using AIPipelineTest. This test checks whether FaceDetector can load the YuNet model, whether FaceRecognizer can load the SFace model, whether detection can run on a test frame without crashing, and whether cosine similarity can distinguish between similar and different embeddings. The test result shows that the main AI pipeline can work before being integrated into the user interface.

The third test was the registration function. The user enters a name, captures an image from the webcam, and saves it. If no face is detected, the program asks the user to capture again. If a face is detected, SFace extracts the embedding and the system saves a GalleryEntry object into gallery.dat. After restarting the application, the saved person still appears in the gallery, which proves that ObjectOutputStream and ObjectInputStream work correctly.

The fourth test was the real-time recognition function. In the Recognize tab, the system detects the face from the live webcam, extracts the embedding, compares it with all registered embeddings, and displays the best result. If the similarity score is lower than 0.363, the result is UNKNOWN. If the score is equal to or higher than 0.363, the system displays the name of the best matched person together with the confidence percentage.

The following figures show two experimental cases of the recognition function. The first case shows a registered user recognized successfully, while the second case shows an unregistered user displayed as UNKNOWN.

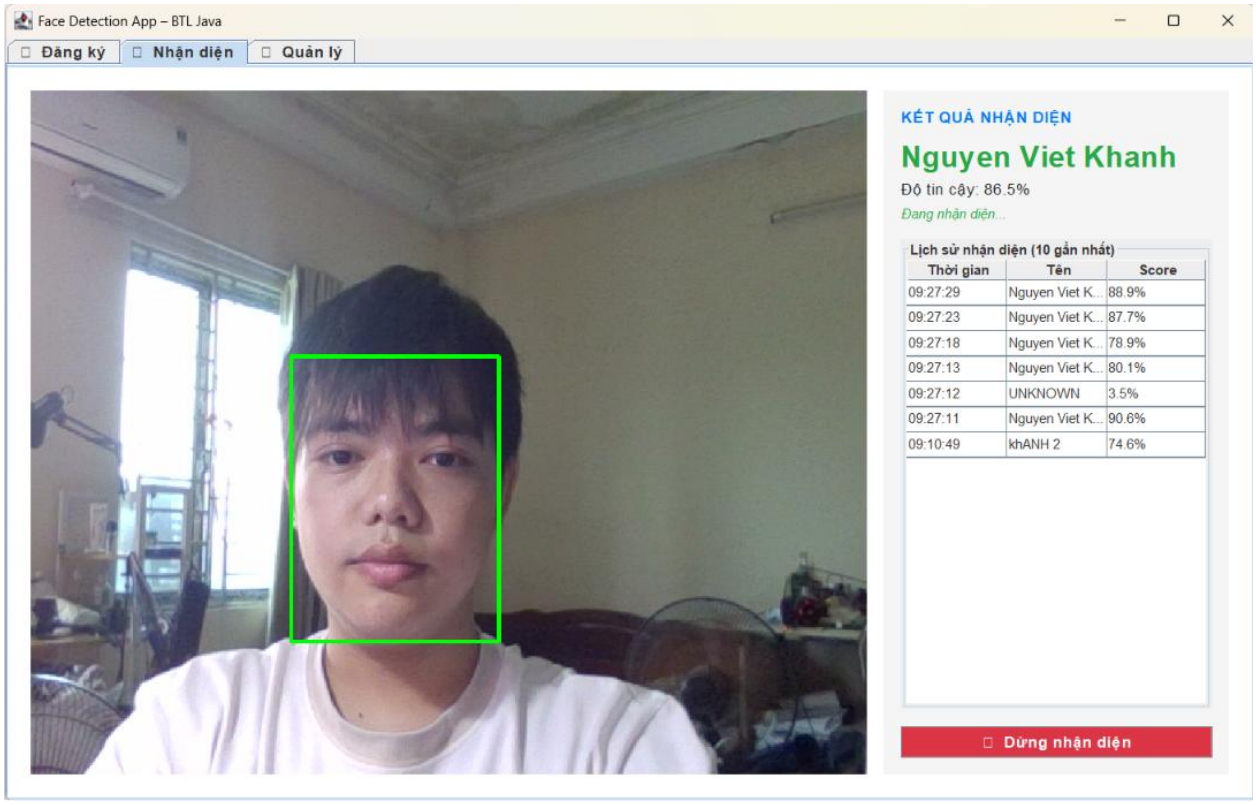


Figure 4.19. Successful Face Recognition Result

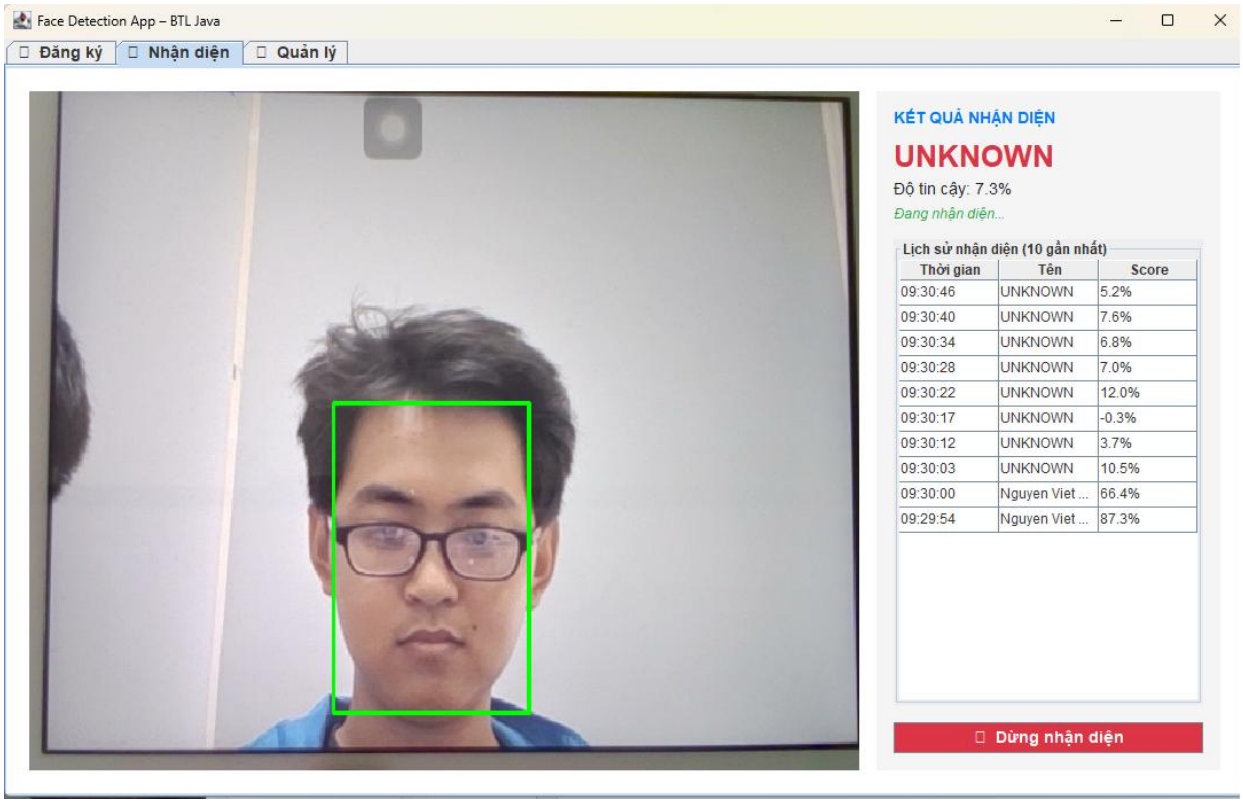


Figure 4.20. Unknown Face Recognition Result

The fifth test was the gallery management function. The Manage tab displays all registered people in a table with the order number, name, and embedding dimension. The refresh button reloads the gallery.dat file. The delete button removes the selected person, and the clear-all button removes all registered entries. After each operation, the file is saved again so that the change remains after restarting the program.

The sixth test was the database connection test. The DatabaseConnection main method was used to check whether the application could connect to MySQL and prepare the detection_log table. When XAMPP MySQL was running and the database existed, the console displayed connection success. This confirms that the JDBC URL, username, password, and table creation statement are suitable for the local testing environment.

The seventh test was the DAO test. DetectionLogDAO inserts sample logs such as Nguyen Van A and Unknown, then reads all logs and reads logs by person name. The returned records are printed to the console in descending time order. This test proves that INSERT, SELECT ALL, and SELECT BY PERSON operations can work with the detection_log table.

In the final integrated test, registration and recognition were tested together with database logging. When a new face was saved, a registration log was inserted into the database. When a registered person was recognized, the recognition result and score were saved into the detection_log table. The application also limited repeated logs for the same person to once every five seconds, so the database did not fill with too many duplicated records.

Through these tests, the system achieved the main requirements of the project. It can open the webcam, detect faces with YuNet, recognize faces with SFace, save registered face embeddings, manage the gallery, and record recognition history in MySQL. Some limitations still exist, such as lower accuracy in poor lighting, wrong face angles, or when the gallery contains too few samples for each person. However, within the scope of a Java final project, the application works correctly and demonstrates the integration of Java Swing, OpenCV, file storage, and JDBC database processing.

CHAPTER 5: CONCLUSION AND FUTURE DEVELOPMENT

5.1 Conclusion

In this project, the group developed a basic face recognition desktop application using Java Swing and OpenCV. The application can open a webcam, detect faces using YuNet, extract facial features using SFace, register new users, recognize registered users in real time, manage stored face data, and save recognition history.

Through the implementation process, the group applied several important Java Technology topics, including object-oriented programming, Java Swing interface design, event handling, file input/output, multithreading, OpenCV integration, and JDBC database connection. The project also helped the group understand how an AI model can be integrated into a normal desktop application.

Although the system is still a basic version, it meets the main requirements of the final project. The program demonstrates a complete workflow from camera input to face detection, feature extraction, comparison, result display, and data storage.

5.2 Limitations of the Project

The system still has some limitations. First, the recognition accuracy depends strongly on lighting conditions, camera quality, and face angle. The system works better when the face is frontal and the lighting is stable. If the face is too dark, too far from the camera, or partially covered, the detection and recognition results may be less accurate.

Second, the gallery currently stores only one embedding for each registration. If the same person is registered with only one face sample, the system may not recognize that person well in different conditions. More samples for each person would help improve stability.

Third, the current system mainly supports a local desktop environment. It does not yet support network deployment, account login, role-based permissions, or advanced attendance reports. The database is also used mainly for logging and has not been expanded into a full user management system.

5.3 Future Development

In the future, the application can be improved in several directions. First, the system should support storing more than one face sample for each person. With multiple embeddings per user, the recognition result can be more stable under different lighting conditions, facial expressions, and camera angles.

Second, the database can be expanded from a simple detection_log table into a complete management database. For example, a users table can store user ID, full name, class, student code, and department. A sessions table can store attendance sessions, and a recognition_log table can connect users with recognition time and device information. This would make the project more suitable for a real attendance system.

Third, the application can provide a log management interface. Users should be able to view recognition history directly inside the program, search logs by person name, filter logs by date, and export results to Excel or PDF. This feature would make the database part more useful for teachers, administrators, or project evaluators.

Fourth, the system can improve security and reliability by adding liveness detection. This feature can help prevent fake recognition using printed photos or phone screens. The system may also add password protection for the management screen and backup functions for gallery.dat and the database.

Finally, the interface can be improved to look more modern and user-friendly. The system can add clearer instructions, better error messages, camera settings, threshold configuration, and a dashboard showing the number of registered users and recent recognition results.

5.4 Lessons Learned

After completing the project, the group gained practical experience in developing a complete Java desktop application. The group learned how to divide the program into packages and classes, how to process webcam images, how to use AI models through OpenCV, and how to store data using both files and databases.

The project also helped improve teamwork and debugging skills. Because the application includes many modules, the group had to test each part carefully and then integrate them into one working system. This experience is useful for future software development projects.

REFERENCES

[1] Oracle, *Java Platform, Standard Edition Documentation*. Oracle. Available: https://docs.oracle.com/en/java/javase/

[2] Oracle, *The Java Tutorials: Getting Started with Swing*. Oracle. Available: https://docs.oracle.com/javase/tutorial/uiswing/start/index.html

[3] Oracle, *The Java Tutorials: JDBC Database Access*. Oracle. Available: https://docs.oracle.com/javase/tutorial/jdbc/index.html

[4] OpenCV, *OpenCV Documentation*. OpenCV. Available: https://docs.opencv.org/4.x/

[5] OpenCV, *VideoCapture Class Reference*. OpenCV. Available: https://docs.opencv.org/4.x/javadoc/org/opencv/videoio/VideoCapture.html

[6] OpenCV, *FaceDetectorYN Class Reference*. OpenCV. Available: https://docs.opencv.org/4.x/df/d20/classcv_1_1FaceDetectorYN.html

[7] OpenCV, *FaceRecognizerSF Class Reference*. OpenCV. Available: https://docs.opencv.org/4.x/da/d09/classcv_1_1FaceRecognizerSF.html

[8] OpenCV, *OpenCV Zoo: Model Zoo for OpenCV DNN and Benchmarks*. GitHub. Available: https://github.com/opencv/opencv_zoo

[9] MySQL, *MySQL Reference Manual: CREATE TABLE Statement*. Oracle. Available: https://dev.mysql.com/doc/en/create-table.html

[10] MySQL, *MySQL Reference Manual: Data Types*. Oracle. Available: https://dev.mysql.com/doc/en/data-types.html

[11] Apache Friends, *XAMPP Documentation*. Apache Friends. Available: https://www.apachefriends.org/docs/

[12] phpMyAdmin, *phpMyAdmin Documentation*. phpMyAdmin. Available: https://www.phpmyadmin.net/docs/